

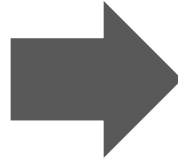
# NotifierProvider

---

- a provider that allows you to expose and listen to a Notifier subclass
  - StateNotifierProvider → StateNotifier, ChangeNotifierProvider → ChangeNotifier
- for handling complex business logic
  - recommended approach
- the state provided by NotifierProvider can only be changed through the interface provided by the Notifier
- The state of Notifier is expected to be initialized synchronously.
  - For asynchronous initializations, see AsyncNotifier.
- can be created using riverpod\_generator

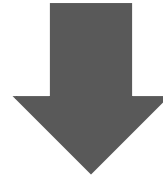
# function tear-offs

```
ElevatedButton(  
  onPressed: () {  
    doSomething();  
  },  
  child: Text('Do something'),  
);
```



```
ElevatedButton(  
  onPressed: doSomething,  
  child: Text('Do something'),  
);
```

```
[ 'my', 'code' ].forEach((element) => print(element));
```



```
[ 'my', 'code' ].forEach(print);
```

# constructor tear-offs

```
Column(  
  children: ['my', 'code'].map((title) => Text(title)).toList(),  
)
```



```
Column(  
  children: ['my', 'code'].map(Text).toList(),  
)
```



```
Column(  
  children: ['my', 'code'].map(Text.new).toList(),  
)
```

```
final counter = ref.watch(counterProvider);
```

```
onPressed: () {  
    ref.read(counterProvider.notifier).increment();  
},
```